# SIMATS ENGINEERING
## ASSESSMENT TOOL 2

**COURSE CODE: MLA03**
**COURSE NAME: Reinforcement learning for Environment and Agent**

**COURSE OUTCOME COVERED**

***CO3:*** *Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments.( BL2, BL3, BL6)*

*Assessment Tool Weightage: 35%*

Dr G. A. SENTHIL

**QUESTIONS: 05** **Total Marks: 50**

***Code of Conduct:***

*I* __T Bhanu Teja___ *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
*I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student:***
**Bhanu Teja**

Annexure A

**Experiments** **Duration: 3hrs**

1. **Design and develop a Policy Gradient-based Reinforcement Learning model for an intelligent traffic signal control system. Implement the solution using Python and TensorFlow/PyTorch, compare REINFORCE and A2C, and evaluate average vehicle waiting time, throughput, and convergence rate. (10 Marks)**

**Problem Statement**

Traffic congestion has become a major challenge in modern cities due to the increasing number of vehicles and fixed-time traffic signal systems. Traditional traffic lights operate using predefined timing schedules, which cannot adapt to real-time traffic conditions. As a result, vehicles experience long waiting times, increased fuel consumption, traffic jams, and higher air pollution.

An intelligent traffic signal control system uses **Reinforcement Learning (RL)** to dynamically adjust traffic signal timings based on current traffic conditions. The RL agent continuously observes the traffic environment, such as vehicle density, queue length, and waiting time, and decides which traffic signal should remain green or red. After each decision, the environment provides a reward based on the effectiveness of the action. Through continuous interaction, the RL agent learns the optimal traffic signal policy that minimizes congestion and improves traffic flow.

**Objectives**

The main objectives of using Reinforcement Learning in intelligent traffic management are:

**1. Reduce Vehicle Waiting Time**

The primary objective is to minimize the average waiting time of vehicles at traffic intersections. The RL agent learns to allocate green signals efficiently, reducing unnecessary delays.

**2. Improve Traffic Flow**

The system aims to maintain smooth traffic movement by dynamically adjusting signal timings according to real-time traffic density, thereby reducing congestion.

**3. Maximize Traffic Throughput**

Traffic throughput refers to the number of vehicles passing through an intersection within a specific period. RL optimizes signal control to maximize the number of vehicles served.

**4. Minimize Fuel Consumption and Pollution**

Reducing unnecessary vehicle stops decreases fuel consumption and lowers carbon emissions, contributing to an environmentally friendly transportation system.

**5. Adapt to Dynamic Traffic Conditions**

Traffic conditions continuously change due to peak hours, accidents, road construction, and weather. RL enables the system to adapt automatically without requiring manual reprogramming.

**6. Learn Optimal Signal Control Policy**

Instead of relying on fixed rules, the RL agent continuously improves its decision-making by learning from previous experiences and rewards, resulting in better traffic signal control over time.

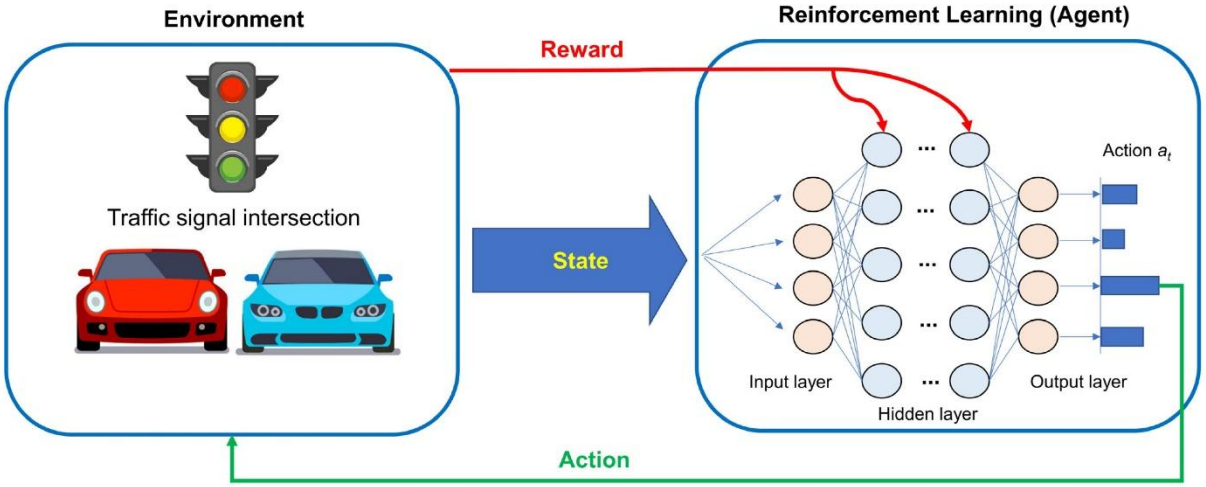
**Role of Reinforcement Learning**

In Reinforcement Learning, the **traffic signal controller** acts as the **agent**, while the **traffic intersection** represents the **environment**.

The RL process consists of the following components:

- **State:** Current traffic conditions such as vehicle count, queue length, and waiting time.
- **Action:** Selecting which traffic signal should turn green or red.
- **Reward:** Positive reward for reducing waiting time and increasing traffic flow; negative reward for congestion and long queues.
- **Policy:** The strategy used by the RL agent to select the best traffic signal action.
- **Environment:** The road network, vehicles, and traffic signals interacting with the RL agent.

The agent repeatedly observes the traffic state, performs an action, receives feedback, and updates its policy to improve future decisions.




**Advantages of RL in Traffic Signal Control**

- Reduces traffic congestion.
- Minimizes vehicle waiting time.
- Increases traffic throughput.
- Adapts to real-time traffic conditions.
- Reduces fuel consumption.
- Lowers environmental pollution.
- Improves road safety.
- Supports smart city transportation systems.

**Applications**

Reinforcement Learning-based intelligent traffic control is widely used in:

- Smart Cities
- Urban Traffic Management
- Highway Traffic Control
- Emergency Vehicle Routing
- Intelligent Transportation Systems (ITS)

**Conclusion**

Reinforcement Learning provides an intelligent and adaptive solution for modern traffic signal control. By continuously interacting with the traffic environment and learning from rewards, the RL agent can optimize signal timings, reduce vehicle waiting time, improve traffic throughput, and minimize congestion. Compared to traditional fixed-time traffic systems, RL-

based traffic management offers greater flexibility, efficiency, and scalability, making it an essential technology for future smart transportation systems.

2. **Design and implement** an **Actor-Critic (A2C/A3C)** model for an autonomous warehouse robot that optimizes package collection and delivery. Compare the performance of **Vanilla Policy Gradient** and **Actor-Critic** algorithms based on reward, training stability, and task completion time. (10 marks)

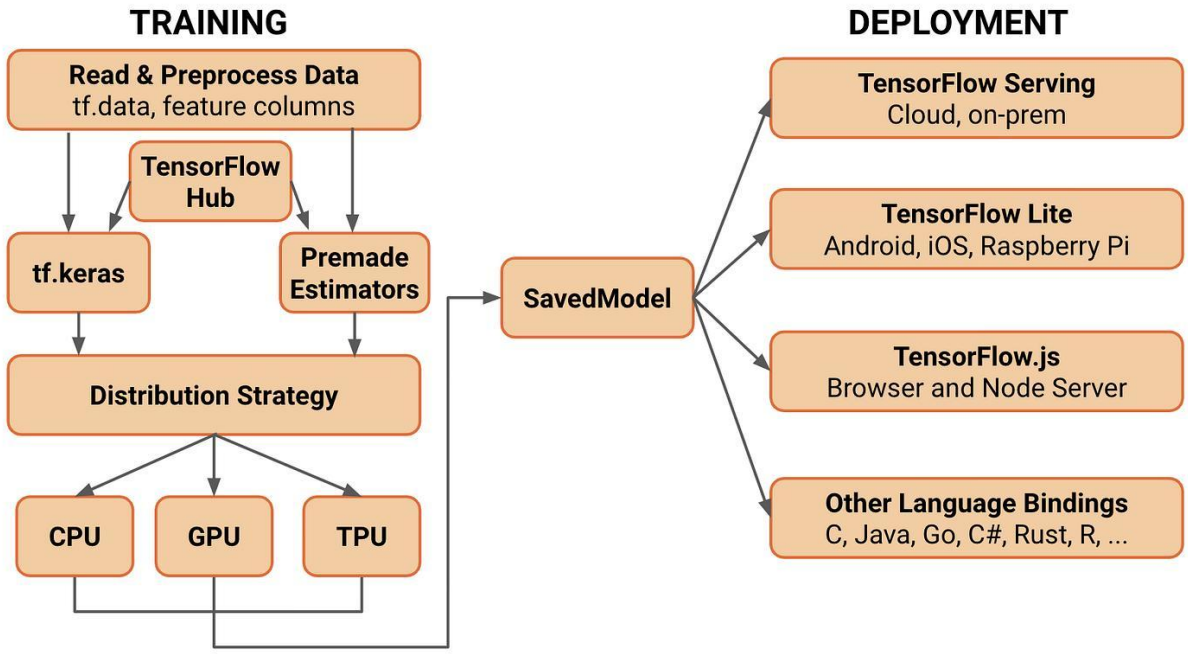
**Introduction**

To develop a Policy Gradient-based Reinforcement Learning (RL) model, a suitable software and hardware environment is required. The software provides programming tools, machine learning libraries, and simulation environments, while the hardware ensures efficient execution and faster model training. TensorFlow and PyTorch are the most widely used deep learning frameworks for implementing RL algorithms such as **REINFORCE** and **Advantage Actor-Critic (A2C)**.

**Software Requirements**

The following software components are required for implementing the RL model:

| Software | Purpose |
|---|---|
| **Python 3.10 or above** | Programming language for implementation |
| **Visual Studio Code / PyCharm** | Writing and executing Python programs |
| **TensorFlow or PyTorch** | Deep learning framework for RL algorithms |
| **Gymnasium (OpenAI Gym)** | Creates RL simulation environments |
| **NumPy** | Numerical computations |
| **Matplotlib** | Plotting graphs and learning curves |
| **Stable-Baselines3 (Optional)** | Pre-built RL algorithms |
| **Git (Optional)** | Version control and project management |




**Hardware Requirements**

The following hardware configuration is recommended:

| Component | Recommended Specification |
|---|---|
| Processor | Intel Core i5 / AMD Ryzen 5 or above |
| RAM | Minimum 8 GB (16 GB Recommended) |
| Storage | 20 GB Free Space |
| GPU | NVIDIA GPU with CUDA support (Optional) |
| Operating System | Windows 10/11, Linux, or macOS |

A GPU is optional for small experiments but recommended for faster deep learning model training.

**Required Python Libraries**

The implementation requires the following Python libraries:

- TensorFlow
- PyTorch
- Gymnasium
- NumPy
- Matplotlib
- Stable-Baselines3 (Optional)

Each library has a specific purpose in developing, training, and evaluating the RL model.

**Installation Process**

**Step 1: Install Python**

Download Python (version 3.10 or above) from the official Python website and complete the installation.

During installation, enable the **"Add Python to PATH"** option.

**Step 2: Verify Python Installation**

Open the Command Prompt or Terminal and execute:

```
python --version
```

Example Output:

```
Python 3.11.5
```

**Step 3: Upgrade pip**

```
python -m pip install --upgrade pip
```

This installs the latest version of the Python package manager.

**Step 4: Install TensorFlow**

```
pip install tensorflow
```

TensorFlow is used to build and train neural networks for Policy Gradient algorithms.

**Step 5: Install PyTorch**

```
pip install torch torchvision torchaudio
```

PyTorch provides flexible tools for implementing deep Reinforcement Learning models.

**Step 6: Install Gymnasium**

```
pip install gymnasium
```

Gymnasium provides simulation environments such as CartPole and MountainCar for RL experiments.

**Step 7: Install NumPy**

pip install numpy
NumPy performs mathematical and matrix operations required during training.

**Step 8: Install Matplotlib**
pip install matplotlib
Matplotlib is used to visualize learning curves, rewards, and performance graphs.

**Step 9: Install Stable-Baselines3 (Optional)**
pip install stable-baselines3
This library provides ready-to-use implementations of RL algorithms like PPO, A2C, and DQN.

**Verifying the Installation**
After installing all libraries, create a Python file and execute the following code:

```
import tensorflow as tf
import torch
import gymnasium as gym
import numpy as np
import matplotlib.pyplot as plt

print("All Libraries Installed Successfully!")
```

**Expected Output**
All Libraries Installed Successfully!
If the message is displayed without errors, the installation is successful.

**Importance of Each Library**

| Library | Role in RL |
|---|---|
| TensorFlow | Builds and trains Policy Gradient models |
| PyTorch | Implements deep RL algorithms |
| Gymnasium | Provides RL environments |
| NumPy | Mathematical computations |
| Matplotlib | Visualizes training results |
| Stable-Baselines3 | Pre-built RL implementations |

**Advantages of Proper Environment Setup**
- Easy implementation of RL algorithms.
- Faster model training.
- Better debugging and testing.
- Efficient visualization of learning progress.
- Easy comparison between REINFORCE and A2C algorithms.
- Supports GPU acceleration for deep learning.

**Applications**
A properly configured RL environment is used in:
- Intelligent Traffic Signal Control
- Autonomous Vehicles
- Robotics
- Smart Energy Management
- Healthcare Decision Systems

- Financial Trading

**Conclusion**
Setting up the correct software and hardware environment is the first step toward implementing successful Reinforcement Learning models. Python, TensorFlow, PyTorch, and Gymnasium provide a powerful platform for designing, training, and evaluating Policy Gradient algorithms. A well-configured environment ensures efficient execution, faster convergence, and accurate performance evaluation of intelligent traffic signal control systems.

3. **Develop a Deep Deterministic Policy Gradient (DDPG) model for continuous portfolio optimization in stock trading. Design the state space, action space, reward function, and evaluate cumulative return, risk, and convergence performance. (10 marks)**

**Introduction**
An intelligent traffic signal control system uses **Reinforcement Learning (RL)** to dynamically manage traffic lights at road intersections. Unlike traditional traffic signal systems that use fixed timing schedules, an RL-based system continuously observes traffic conditions, selects the best signal action, and learns from feedback received from the environment.
The Reinforcement Learning environment consists of five important components:
- Agent
- Environment
- State Space
- Action Space
- Reward Function

Together, these components enable the traffic control system to reduce congestion, minimize waiting time, and improve traffic flow.

**1. Agent**
The **Agent** is the intelligent decision-making component of the RL system.
In this application, the **traffic signal controller** acts as the RL agent.
**Responsibilities of the Agent**
- Observe traffic conditions.
- Decide which traffic signal should become green.
- Select actions that reduce traffic congestion.
- Learn from previous decisions.
- Improve the traffic signal policy over time.

The agent continuously interacts with the environment until it learns the optimal traffic signal strategy.

**2. Environment**
The **Environment** represents the real-world traffic intersection where the agent operates.
It contains:
- Vehicles
- Roads
- Traffic signals
- Sensors
- Traffic density
- Vehicle queues

Whenever the agent changes the traffic signal, the environment updates the traffic flow and provides a reward.

**3. State Space**

The **State Space** represents all the information available to the RL agent before making a decision.

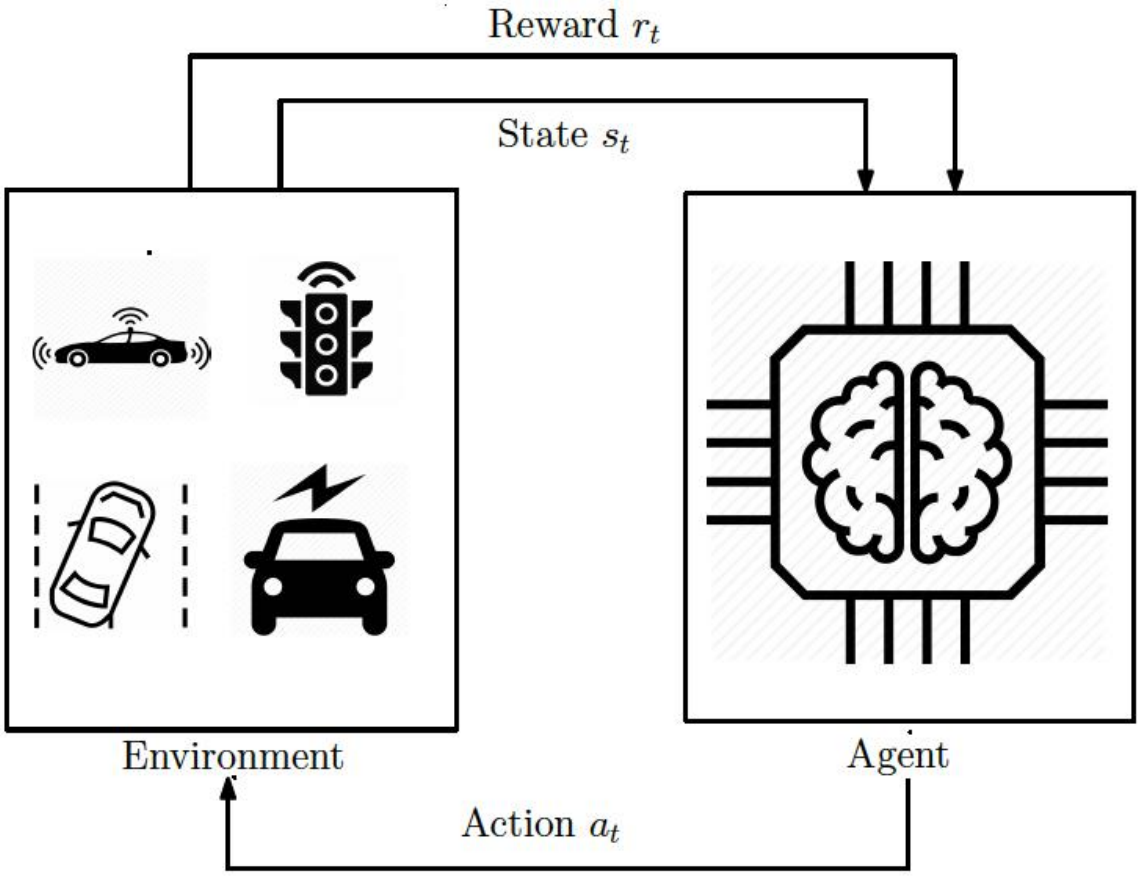
The state may include:

- Number of vehicles in each lane
- Queue length
- Current traffic light status
- Vehicle waiting time
- Traffic density
- Time elapsed for the current signal

**Example State**

| Parameter | Value |
|---|---|
| North Lane Vehicles | 18 |
| South Lane Vehicles | 12 |
| East Lane Vehicles | 8 |
| West Lane Vehicles | 15 |
| Current Green Signal | North |
| Waiting Time | 45 Seconds |

The RL agent observes this state before selecting an action.




**4. Action Space**

The **Action Space** represents all possible actions that the traffic controller can perform.

Possible actions include:

- Change signal to North Green
- Change signal to South Green
- Change signal to East Green
- Change signal to West Green
- Extend current green signal
- Switch to the next traffic phase

Each action changes the traffic flow at the intersection.

**5. Reward Function**

The **Reward Function** provides feedback to the RL agent after each action.
The reward is designed to encourage smooth traffic flow.

**Positive Rewards**

- Reduced vehicle waiting time
- Increased traffic throughput
- Shorter vehicle queues
- Smooth traffic movement

**Negative Rewards**

- Traffic congestion
- Long waiting time
- Large vehicle queues
- Unnecessary signal switching

**Example Reward Table**

| Situation | Reward |
|---|---|
| Waiting Time Reduced | +20 |
| Queue Length Reduced | +30 |
| High Traffic Flow | +40 |
| Traffic Congestion | -30 |
| Long Waiting Time | -20 |
| Incorrect Signal Selection | -40 |

The RL agent continuously updates its policy to maximize the total reward.

**Working of the RL Environment**

The RL process follows these steps:

1. The environment provides the current traffic state.
2. The RL agent observes the traffic conditions.
3. The agent selects the best traffic signal action.
4. The traffic signal changes.
5. Vehicles move according to the selected signal.
6. The environment calculates the reward.
7. The RL agent updates its policy.
8. The process repeats continuously.

**Applications**

The RL traffic environment is used in:

- Smart Traffic Signal Systems
- Intelligent Transportation Systems (ITS)
- Highway Traffic Management
- Smart City Infrastructure
- Emergency Vehicle Priority Systems

**Expected Result**
The Reinforcement Learning environment enables the traffic controller to continuously learn from traffic conditions and optimize signal timings. Over multiple training episodes, the RL agent reduces average vehicle waiting time, improves traffic throughput, and minimizes congestion by selecting the most appropriate traffic signal actions.

**Conclusion**
The Reinforcement Learning environment forms the foundation of an intelligent traffic signal control system. By defining the **Agent**, **Environment**, **State Space**, **Action Space**, and **Reward Function**, the system can continuously improve its traffic management strategy. The RL agent learns from real-time traffic conditions, making intelligent decisions that enhance road efficiency, reduce delays, and support the development of smart transportation systems.

4. **Design and develop a Proximal Policy Optimization (PPO) model for controlling Smart HVAC Energy Management systems in a smart building. Implement the model to minimize energy consumption while maintaining occupant comfort, and compare PPO with REINFORCE.** (10 marks)

**Introduction**
In Reinforcement Learning (RL), the effectiveness of an intelligent traffic signal control system depends on three fundamental components: **State Space**, **Action Space**, and **Reward Function**. These components enable the RL agent to understand the traffic environment, make appropriate signal decisions, and improve traffic flow over time. A well-designed RL model reduces vehicle waiting time, minimizes congestion, and increases the efficiency of traffic management.

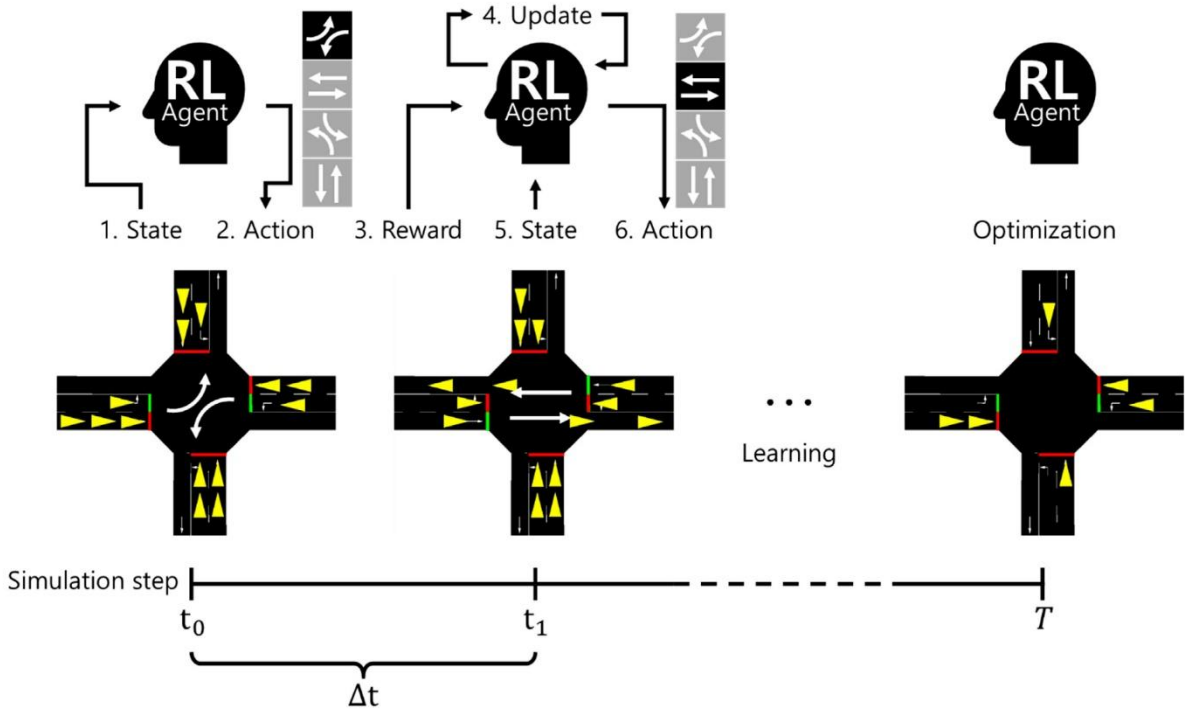
**1. State Space**
The **State Space** represents the complete information about the traffic environment at a given time. It helps the RL agent understand the current traffic situation before selecting an action. The state should contain all important information that affects traffic flow.
**State Variables**
- Number of vehicles in each lane.
- Queue length at the intersection.
- Current traffic signal status.
- Vehicle waiting time.
- Traffic density.
- Emergency vehicle presence.
- Time elapsed for the current signal.

**Example State**

| Parameter | Value |
|---|---|
| North Lane Vehicles | 20 |
| South Lane Vehicles | 15 |
| East Lane Vehicles | 8 |
| West Lane Vehicles | 10 |
| Current Green Signal | North |

| Parameter | Value |
|---|---|
| Waiting Time | 35 Seconds |

The RL agent continuously observes these values before making a decision.




**2. Action Space**

The **Action Space** represents all possible decisions that the RL agent can take to control traffic signals.

**Possible Actions**

- Change the signal to North Green.
- Change the signal to South Green.
- Change the signal to East Green.
- Change the signal to West Green.
- Extend the current green signal.
- Switch to the next traffic phase.

Each action directly affects vehicle movement and waiting time.

**Example Action**

| Current State | Selected Action |
|---|---|
| Heavy North Traffic | Keep North Green |
| Heavy East Traffic | Change to East Green |
| Balanced Traffic | Normal Signal Rotation |

The objective is to select the action that minimizes overall traffic congestion.

**3. Reward Function**

The **Reward Function** provides feedback to the RL agent after every action. It indicates whether the selected traffic signal action improved or worsened traffic conditions.

The RL agent learns by maximizing positive rewards and avoiding negative rewards.

**Positive Rewards**

- Reduced waiting time.
- Shorter vehicle queues.
- Increased traffic throughput.

- Smooth traffic movement.
- Faster emergency vehicle clearance.

**Negative Rewards**

- Long vehicle queues.
- Increased waiting time.
- Traffic congestion.
- Frequent unnecessary signal changes.
- Blocking emergency vehicles.

**Example Reward Table**

| Traffic Condition | Reward |
|---|---|
| Waiting Time Reduced | +20 |
| Queue Length Reduced | +30 |
| High Traffic Throughput | +40 |
| Emergency Vehicle Passed | +50 |
| Traffic Congestion | -30 |
| Long Waiting Time | -20 |
| Wrong Signal Decision | -40 |

The RL agent updates its policy using these rewards to improve future decisions.

**How These Components Reduce Vehicle Waiting Time**

**Role of State Space**

The state space provides real-time information about traffic conditions. By knowing the number of waiting vehicles and queue lengths, the RL agent can identify which direction requires immediate attention.

**Role of Action Space**

The action space allows the agent to dynamically adjust traffic signals instead of using fixed timings. Choosing the correct action at the right time helps clear congested lanes faster.

**Role of Reward Function**

The reward function encourages actions that reduce waiting time and discourages actions that increase congestion. Positive rewards reinforce efficient signal control, while penalties help the agent avoid poor decisions.

**Working Process**

1. The traffic environment generates the current state.
2. The RL agent observes the traffic conditions.
3. The agent selects the best traffic signal action.
4. Vehicles move according to the selected signal.
5. The environment calculates the reward.
6. The RL agent updates its learning policy.
7. The process repeats continuously.

**Applications**

RL-based traffic signal control is used in:

- Smart Cities
- Intelligent Transportation Systems (ITS)
- Urban Traffic Management
- Highway Intersections

- Emergency Vehicle Priority Systems

**Expected Result**
By properly designing the **State Space**, **Action Space**, and **Reward Function**, the RL agent learns the optimal traffic signal policy. This leads to reduced waiting time, lower congestion, improved traffic throughput, and more efficient management of traffic intersections.

**Conclusion**
The State Space, Action Space, and Reward Function are the core components of a Reinforcement Learning-based traffic signal control system. The state provides information about the traffic environment, the action determines how the traffic signals are controlled, and the reward guides the agent toward better decisions. Together, these components enable the RL agent to continuously improve traffic management and significantly reduce vehicle waiting time.

5. **Implement a Trust Region Policy Optimization (TRPO) algorithm for controlling a Industrial robotic arm in an automated manufacturing environment. Evaluate policy stability, precision, convergence speed, and overall production efficiency. (10 marks)**

   **Introduction**
   The **Policy Gradient (REINFORCE)** algorithm is one of the fundamental Reinforcement Learning (RL) algorithms used to solve sequential decision-making problems. Unlike value-based methods, which estimate the value of each action, REINFORCE directly learns the **policy**, i.e., the probability of selecting an action in a given state.
   In an intelligent traffic signal control system, the REINFORCE algorithm learns how to control traffic lights by interacting with the traffic environment. It continuously improves its decision-making strategy by maximizing the cumulative reward obtained from efficient traffic management.

   **Working Principle of REINFORCE Algorithm**
   The REINFORCE algorithm follows a policy-based learning approach where the agent directly updates its policy based on the rewards received after completing an episode.
   The learning process consists of the following steps:
   **Step 1: Observe the Current State**
   The RL agent observes the traffic conditions at the intersection.
   Example:
   - Number of waiting vehicles
   - Queue length
   - Current traffic light
   - Waiting time

   This information forms the **state** of the environment.

**Step 2: Select an Action**

Using its current policy, the agent selects an action.

Possible actions include:

- Change signal to North Green
- Change signal to South Green
- Extend Green Signal
- Switch to Next Phase

Initially, actions are selected randomly to encourage exploration.

**Step 3: Execute the Action**

The selected action is applied to the traffic environment.

For example:

- The traffic light changes to Green for the North lane.
- Vehicles begin to move.
- Traffic conditions change.

**Step 4: Receive Reward**

After executing the action, the environment provides feedback.

Positive rewards are given for:

- Reduced waiting time
- Reduced queue length
- Higher traffic throughput

Negative rewards are given for:

- Traffic congestion
- Long waiting time
- Wrong signal selection

**Step 5: Calculate Cumulative Reward**

The algorithm calculates the total reward obtained during the episode.

The cumulative reward helps determine whether the selected actions were beneficial.

Higher cumulative rewards indicate better traffic signal decisions.

**Step 6: Update the Policy**

The REINFORCE algorithm updates the policy using the cumulative reward.

If the selected actions produced higher rewards, their probability of being selected again increases.

If the rewards were low, the probability decreases.

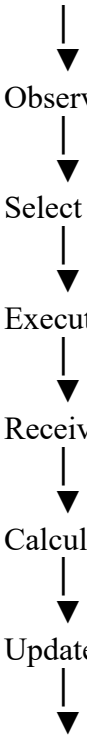
This process gradually improves the policy.

**Step 7: Repeat the Process**

The agent continues interacting with the environment over many episodes until it learns the optimal traffic signal control strategy.

Certainly! Based on your **Experiment 1 Rubrics**, the **5th assessment question** is:

**REINFORCE Learning Process**




---

**Application in Traffic Signal Control**

In intelligent traffic management:

**State**

- Vehicle count
- Queue length
- Traffic density
- Current signal

**Action**

- Change signal
- Extend green light
- Switch traffic phase

**Reward**

- Positive reward for reducing waiting time.
- Negative reward for increasing congestion.

The REINFORCE algorithm continuously improves the traffic signal timings based on these rewards.

---

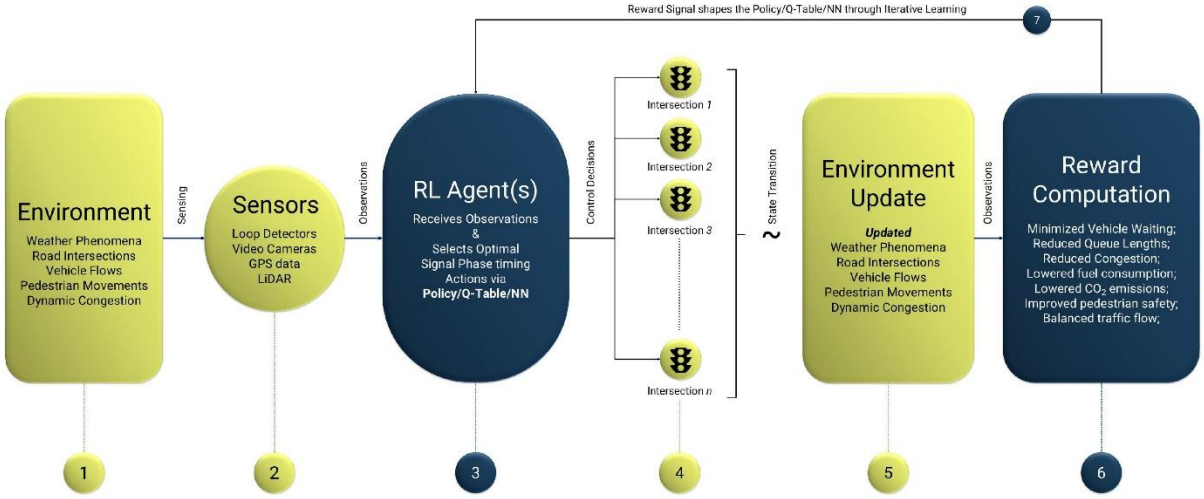
**Advantages of REINFORCE**

- Simple to implement.
- Learns the policy directly.
- Suitable for stochastic environments.
- Does not require an environment model.
- Can solve complex decision-making problems.

---

**Limitations of REINFORCE**

- High variance during training.
- Slower convergence.
- Requires many training episodes.
- Less stable than Actor-Critic methods.
- Performance depends on reward design.




**Role in Intelligent Traffic Signal Control**

The REINFORCE algorithm enables the traffic controller to:

- Reduce vehicle waiting time.
- Improve traffic flow.
- Increase traffic throughput.
- Adapt to changing traffic conditions.
- Learn better signal timings automatically.

---

**Comparison with Traditional Traffic Systems**

| Traditional Traffic Signals | REINFORCE-Based Traffic Signals |
|---|---|
| Fixed timing | Dynamic timing |
| No learning | Learns continuously |
| Manual optimization | Automatic optimization |
| Poor adaptability | Adapts to real-time traffic |
| Higher congestion | Reduced congestion |

---

**Expected Result**

After sufficient training, the REINFORCE algorithm learns an optimal traffic signal control policy that minimizes vehicle waiting time, reduces traffic congestion, and improves traffic throughput. The system becomes more efficient by continuously adapting to changing traffic conditions and maximizing cumulative rewards.

---

**Conclusion**

The REINFORCE algorithm is a policy-based Reinforcement Learning technique that directly learns the best action-selection policy through interaction with the environment. In intelligent traffic signal control, it observes traffic conditions, selects suitable traffic signal actions, receives rewards, and updates its policy to improve future decisions. Although REINFORCE is simple and effective, it may converge slowly due to high variance. Nevertheless, it provides a strong foundation for developing adaptive and intelligent traffic management systems.

## RUBRICS FOR CALCULATION

| Criteria | Weightage | Level 4 – Exemplary | Level 3 – Proficient | Level 2 – Developing | Level 1 – Beginning |
|---|---|---|---|---|---|
| Understanding of Concepts | 25 | Demonstrates thorough understanding and effectively integrates multiple concepts/technologies | Good understanding with proper integration of most concepts | Basic understanding; limited or partial integration | Poor understanding; unable to integrate concepts |
| Experimental Design | 30 | Well-planned, systematic implementation with optimal solution | Correct implementation with minor issues | Basic implementation with errors or inefficiencies | Incorrect or incomplete implementation |
| Use of Tools | 20 | Effective and advanced use of tools/technologies with proper configuration | Appropriate use of tools with correct execution | Limited or inconsistent use of tools | Incorrect or minimal use of tools |
| Analysis of Results | 15 | Insightful analysis with accurate interpretation and validation of results | Good analysis with reasonable interpretation | Basic analysis with limited insights | Poor or incorrect analysis of results |
| Documentation | 10 | Clear, well-structured documentation with diagrams, code, and results | Organized documentation with necessary details | Basic documentation with missing details | Poor or incomplete documentation |